

Journal 02: Introduction to Prompt Engineering

Sharie Rhea

Southern New Hampshire University

CS 530: Artificial Intelligence Principles and Applications

Joshua Andrews

February 15, 2026

Artificial intelligence (AI) has become a powerful tool in the past decade, and advances in computational hardware have accelerated its adoption. However, utilizing AI effectively can be more challenging than it may seem at first glance. Prompt engineering is the practice of tailoring AI input stimuli, in the form of prompts, to achieve high quality output (Liu et al., 2025).

Without a well-thought-out prompt, responses may be vague, irrelevant, overly verbose, misleading, or even flat-out incorrect. This paper will explore the results of using two AI tools—GitHub Copilot and ChatGPT—to evaluate two prompt engineering frameworks and how they affect output quality. The following sections outline the research methodology, examine the strengths and limitations of each tool alongside ethical considerations, and ultimately justify the most effective framework in a software engineering context.

GitHub Copilot and ChatGPT were used for this experiment. Copilot was set to the “ask” mode in Visual Studio Code to avoid responses focused solely on code modification. Two prompt engineering framework acronyms, SIMPLE and SCREAM, were chosen because they are not widely accepted frameworks with strict definitions. The goal of this research is to examine how each tool reacts to different prompt techniques. Each chatbot was first asked a direct question with no prompt engineering. With each iteration, the prompts increasingly incorporated aspects of one of the two frameworks as suggested by the tools themselves. ChatGPT was first asked to explain SCREAM and Copilot to explain SIMPLE. Using these initial responses, follow-on prompts were crafted based on the AIs’ explanations. In other words, the SIMPLE framework was used to ask about SCREAM and vice versa. After a series of prompts, the roles were then reversed for each tool.

Although ChatGPT and Copilot were prompted similarly, it quickly became clear that each tool has distinct strengths and limitations. ChatGPT excelled when it was prompted for written

summaries, producing well-written paragraphs with a nice flow (OpenAI, 2026). On the other hand, Copilot struggled to generate full sentences and frequently relied on repetitive structure and semicolons, almost as if it were writing code (GitHub, 2026). In subsequent prompts, providing a counter example appeared to improve the output quality (GitHub, 2026). Both tools were able to produce bulleted lists following a specified format. Copilot in particular, at least in the “ask” mode, also provides a numbered list of “steps” in almost every output, likely due to tuning done by GitHub (GitHub, 2026). As a result, Copilot may be particularly well suited for project planning or exploring how to approach a problem.

There are also many ethical considerations when utilizing AI tools. In the context of this research, it can be seen that both ChatGPT and Copilot provide authoritative responses when explaining the SIMPLE and SCREAM frameworks. However, they both provided different explanations for each letter. When asked directly if there are other possible definitions, each tool correctly answered with a simple “yes” response (GitHub, 2026; OpenAI, 2026). This suggests that AI often presents a single definitive answer, even when ambiguity exists. Conscientious AI users should recognize this tendency and avoid taking the first response at face value. Beyond ambiguity, another ethical concern involves biases learned from training data. It is likely that ChatGPT and Copilot had different definitions for each framework due to different underlying training data. In software engineering contexts, privacy and data security must also be carefully considered. AI can be a wonderful tool for generating or debugging code, but it is not always clear how the code solves the problem that was prompted. Without a good understanding by a human developer, it is nearly impossible to guarantee secure coding practices.

Both ChatGPT and Copilot have a place in the realm of AI chatbots, and the SIMPLE and SCREAM frameworks can each be used effectively. However, based on the inputs and outputs

from both Copilot and ChatGPT, the SCREAM framework—defined here as Situation, Constraints, Role, Examples, Action, and Metrics—proved most effective in a software engineering context. A situation helps to frame the context of the prompt, which can be useful when altering code in one file that is part of a larger project. Constraints are essential for controlling output length. Both tools generated overly verbose responses to the initial prompt, which can contribute to developer fatigue when sifting for relevant information (GitHub, 2026; OpenAI, 2026). Assigning a role is a great way to alter the tone of the output. For example, code generated from the perspective of an expert cybersecurity professional is likely to be higher quality than code written from the perspective of an undergraduate student. Examples are the best way to ensure the format of the response is as desired. Counter examples are very effective for eliminating undesired behavior. Action defines the task itself, which is of course the most important part of the prompt. Finally, metrics can be a good way to measure success and can increase accuracy. This framework can be applied to any software engineering task, such as generating code, debugging, planning tasks, writing documentation, applying an algorithm, and more.

The largest limitation with applying this framework is that each prompt becomes quite long. Some tools only allow a certain number of messages or tokens at the free tier before cutting off access to responses (GitHub 2026; OpenAI, 2026). For this reason, selectively applying elements of the framework can help reduce prompt length. For example, the situation does not always provide crucial information to the prompt, and there may not be any good examples to provide. Finally, engineers should be cautious and validate the outputs of AI tools, as hallucinations and false authoritative answers may be generated.

References

GitHub. (2026). *Copilot*. <https://github.com/features/copilot/ai-code-editor>

Liu, A. C.-C., Law, O. M. K., & Law, I. (2025). *Machine Learning, Data Science Foundations with Python*. zyBook. <https://learn.zybooks.com/zybook/CS-530-10427.202617-1>

OpenAI. (2026). *ChatGPT* (GPT-5.2). <https://chat.openai.com/chat>